

Reineke-RAG

Technical Handbook

Reineke-RAG — 2026

Reineke-RAG — Technical Handbook

For administrators, DevOps, architects, and implementers. Covers what you need to deploy, operate, extend, and troubleshoot Reineke-RAG on a single-host Docker Compose deployment.

Read before: [TECH_DESCRIPTION.md](#) for the conceptual overview. This handbook assumes you know *what* Reineke-RAG is; it documents *how to run it*.

Authoritative sources inside the repo: [docs/02_ARCHITECTURE.md](#), [docs/04_OPERATIONS.md](#), [docs/adr/](#). This handbook is the working copy — when the two disagree, the docs directory wins and this file gets an update.

Contents

1. [Audience and prerequisites](#)
2. [System overview for operators](#)
3. [Host preparation](#)
4. [Installation \(online\)](#)
5. [Installation \(air-gapped\)](#)
6. [Day-2 operations — rag-admin CLI](#)
7. [User and group management](#)
8. [Folders and ACLs](#)
9. [Document ingestion and lifecycle](#)
10. [Retrieval behaviour and tuning](#)
11. [Model management](#)
12. [Observability](#)
13. [Backup and restore](#)
14. [Security operations](#)
15. [Upgrades](#)
16. [Performance tuning](#)
17. [Troubleshooting](#)
18. [Extending the system](#)
19. [Appendices](#)

1. Audience and prerequisites

This handbook assumes you are comfortable with:

- Docker Compose and container lifecycle (`up` , `logs` , `exec` , `restart`).
- A POSIX shell and basic Linux/macOS system administration.
- OIDC and JWT concepts at a user level (issuer, client, scope, groups claim).

- Basic SQL and the idea of Postgres roles.
- Reading container logs and identifying "which service is failing" from a composite error.

You do **not** need to be a machine-learning engineer. The model routing config is declarative; the retrieval pipeline is not something you edit in production — you tune knobs (k values, chunk size, rerank count).

Minimum knowledge gate to operate unassisted

Before running this system in production, you should be able to:

- Explain what Authentik does and how a user's group ends up in a service's JWT.
- Read a Langfuse trace and point to the reranker span.
- Run `rag-admin backup run` and restore into a throwaway directory.

If any of these feels unfamiliar, step through §13 (Backup and restore) and §12 (Observability) on a staging host first.

2. System overview for operators

2.1 The 25 containers, grouped by concern

Group	Service	Role	Fails to...
Edge	caddy	TLS termination, routing, internal CA	Everything externally
Identity	authentik-server, authentik-worker, authentik-db, authentik-redis	OIDC IdP + groups + blueprints	Logins broken; emergency token fallback
App DB	postgres	rag.* schema	APIs return 5xx
Queue	redis	RQ queue, pub/sub	Ingestion stalls
Objects	minio	Raw files (immutable, versioned)	Uploads/previews broken
Vectors	qdrant	Hybrid index + ACL filter	Retrieval broken
LLM runtime	ollama, ollama-init, tei-reranker	Generation + embed + rerank	Answers broken
Custom	docling, ingestion-api, ingestion-worker, retrieval-api, duckdb-api	The product	Functionality broken
UI	openwebui, pipelines	End-user chat	Users see a generic OIDC error
Observability	langfuse, langfuse-db, prometheus, grafana, loki, promtail	Traces + metrics + logs	Debugging harder; product keeps running
Automation (profile: automation)	n8n, watcher	Scheduled jobs + folder watch	Automation stops

2.2 Host-facing surface

Only **80** and **443** are published by Docker on the host. Everything else is reached via service names on the internal bridge network `reineke`.

External paths through Caddy:

URL	Target
/	Open WebUI
/auth/...	Authentik
/langfuse	Langfuse
/grafana	Grafana
/n8n/	n8n (automation profile)

Internal service ports live in [docs/04_OPERATIONS.md Appendix A](#).

2.3 Persistence surface

One data root (`${DATA_ROOT}` , default `/var/lib/reineke`) contains **one subdirectory per service**. That keeps the backup script legible: one directory → one procedure.

```
postgres/  authentik-db/  langfuse-db/  redis/  minio/  qdrant/
duckdb/    ollama/        tei/          docling/ loki/    grafana/
```

3. Host preparation

3.1 Hardware

Resource	Minimum	Reference	Recommended
CPU / SoC	8 cores	Apple M4 Max (14 cores)	M4 Max or Linux x86 + 24 GB GPU
RAM	32 GB	64 GB	64–128 GB
Disk (SSD)	250 GB	1 TB	1 TB + external backup
OS	macOS 14+ / Ubuntu 22.04+	macOS 15 on M4 Max	—
Container runtime	Docker 24+ or Colima 0.7+	Colima 0.7	—

On macOS, use Colima with a generous resource allocation:

```
colima start --cpu 10 --memory 48 --disk 200
```

Leave ≥ 16 GB of RAM for macOS itself. Docker Desktop is fine; give it the same budget.

3.2 DNS and TLS

- Add an internal DNS entry `rag.<your-company>.local` → host IP.
- TLS by default uses Caddy's internal CA. Export the CA certificate with `scripts/export-ca.sh > ca.crt` and distribute it to clients so browsers do not warn.
- For Let's Encrypt via DNS-01, edit the Caddyfile during build handover with the deployment-agent.

3.3 Ports

Only **80** and **443** inbound on the host. *All other Docker ports must remain on the bridge*. If your host has a firewall, reject everything else except SSH.

3.4 Directories

```
mkdir -p /var/lib/reineke /etc/reineke
sudo chown "$USER" /var/lib/reineke /etc/reineke
```

3.5 Secrets

`scripts/bootstrap.sh` generates missing secrets and writes them to `.env`. Before first run, decide on and collect (to a password manager):

- `AUTHENTIK_BOOTSTRAP_PASSWORD` (admin's first-login password, will be forced-changed)
- `BACKUP_GPG_PASSPHRASE_FILE` (optional; for encrypted backups)
- `ALERT_WEBHOOK_URL` (optional)

Everything else (`POSTGRES_PASSWORD`, `QDRANT_API_KEY`, `MINIO_ROOT_PASSWORD`, `INTERNAL_SERVICE_TOKEN`, `LANGFUSE_*`, `AUTHENTIK_SECRET_KEY`) can be auto-generated. After generation, **store `.env` itself** in your password manager as a single attachment.

4. Installation (online)

Once `services/**` has been produced by the agent build, first-time install is:

```
# 1. Clone or extract the release
cd /opt && tar xf reineke-rag-v1.0.0.tar.gz && cd reineke-rag

# 2. Generate .env + owner-inputs.yaml
bash scripts/bootstrap.sh

# 3. Pull container images (online)
make pull

# 4. Pull LLM + embedder + reranker models (~80 GB with heavy tier)
make pull-models                # or: PULL_HEAVY=false make pull-models

# 5. Start the core stack
make up

# 6. Wait for healthchecks (2–5 min on first start)
make wait-healthy

# 7. Open Authentik and complete bootstrap
open https://$PRIMARY_DOMAIN/auth/

# 8. Seed folders + ACLs from config/owner-inputs.yaml
rag-admin folders sync config/owner-inputs.yaml

# 9. Smoke-test retrieval
rag-admin query "Ping"           # expects refusal; proves auth + retrieval path
```

The smoke-test is deliberately expected to refuse — a "can't find information" response with no leakage proves that (a) OIDC works, (b) Qdrant returned zero points for an unknown term, (c) refusal style triggers correctly.

4.1 Compose profiles

Profile	Purpose	Invocation
(default)	Core 19 services	<code>make up</code>
<code>minimal</code>	Core minus Langfuse/Loki/Grafana	<code>PROFILES=minimal make up</code>
<code>automation</code>	Core + n8n + watcher	<code>make up-automation</code>
<code>init</code>	One-shot: pulls Ollama models and exits	<code>make pull-models</code>

5. Installation (air-gapped)

The stack is designed to run without outbound network access. A helper online machine builds a ~100 GB bundle:

```
# On the online helper
bash scripts/pack-offline.sh
# Produces reineke-rag-offline-<date>.tar.gz containing:
# - docker save of every image (pinned tags)
# - Ollama model weights (bge-m3, gemma2:9b, qwen2.5:32b, llama3.3:70b)
# - TEI reranker weights
# - Docling OCR models
# - Python wheels for custom services
```

On the target:

```
bash scripts/load-offline.sh reineke-rag-offline-<date>.tar.gz
make up
make wait-healthy
```

Every runtime path (retrieval, ingestion, auth) must succeed with the LAN cable unplugged — this is tested in Phase 9 (A9.3). If outbound calls appear in the logs, that is a bug.

6. Day-2 operations — `rag-admin` CLI

`rag-admin` is a thin Python wrapper that authenticates as an admin and hits the three custom APIs. Most frequent commands:

```

# Status + health
rag-admin status                # container health, queue depth, doc count
rag-admin jobs list --state failed

# Users
rag-admin users list
rag-admin users add alice@company.de --groups engineering,qms

# Folders + ACLs
rag-admin folders list
rag-admin folders create /qms/normen --groups admin,qms,engineering
rag-admin folders set    /qms/normen admin,qms,engineering,auditor
rag-admin folders move   /qms/normen /qms/standards          # rewrites doc rows
rag-admin folders delete /qms/normen --wait                  # refuses if docs exist

# Documents
rag-admin docs list --folder /qms/normen
rag-admin docs upload ./drop/*.pdf --folder /qms/normen
rag-admin docs ingest-dir drop/ --base-folder /              # recursive
rag-admin docs reindex <doc_id>
rag-admin docs reindex --folder /qms/normen
rag-admin docs reindex --all --confirm                        # nuclear
rag-admin docs delete <doc_id>                                # soft; 30-day retention
rag-admin docs delete <doc_id> --hard                         # requires ADMIN_CONFIRM=yes

# Queries (for testing)
rag-admin query "Welche Norm gilt für Typ-B-Schränke?"

# Models
rag-admin models list

# Backups
rag-admin backup run
rag-admin restore plan <backup-dir>    # dry run
rag-admin restore apply <backup-dir>

# Sessions + audit
rag-admin sessions revoke-all
rag-admin audit export --from 2026-01-01 --to 2026-03-31 --format csv > q1.csv

# Alerts
rag-admin alerts silence <rule> --until 2026-05-01

```

Authoritative list in [docs/04_OPERATIONS.md §2.3](#). Every command hits an HTTP API; anything you can do with the CLI you can do by calling `ingestion-api` or `retrieval-api` directly with an admin JWT.

7. User and group management

Users and groups are managed in **Authentik**. The app DB mirrors the minimum (`rag.users.id` , email, groups) for audit joins; the mirror refreshes on every JWT validation.

7.1 Adding a user

1. `https://$PRIMARY_DOMAIN/auth/` → Directory → Users → Create.
2. Fill name + email. Leave password empty to send an invite email (if SMTP configured); otherwise set a temporary password manually and communicate out-of-band.
3. Directory → Groups → pick group(s) → add user as member.
4. On next login, the user lands in the chat UI at `/` with the right permissions.

7.2 Adding a group

1. Directory → Groups → Create. Pick the name that will be used in folder ACLs (e.g. `auditor`).
2. `rag-admin folders set /<path> <groups>` to grant it access to folders.
3. Watch `rag-admin jobs list --kind reac1 drain` (≤ 30 s per 1 000 chunks).

7.3 Offboarding

1. In Authentik, toggle the user to Inactive.
2. Access tokens (15 min) and refresh tokens (24 h) expire naturally; effective lockout < 1 min in practice because the next token refresh fails.
3. Chats are retained per policy; audit entries per retention rules. For an anonymisation path, see the privacy-notes workstream (draft).

7.4 Password reset

- Via Authentik's self-service flow if SMTP is configured.
- Otherwise: Directory → Users → Reset password (temporary) → deliver out-of-band.

7.5 Emergency: Authentik down

`ADMIN_BACKUP_TOKEN` in `.env` is a long-lived, emergency-only JWT that grants admin access to `rag-admin` and the APIs. Rotate every 90 days. Use is logged loudly in `audit_log`. Never use for normal operations.

8. Folders and ACLs

The folder tree is **logical** (a database table), not a filesystem. `rag.folders` is the source of truth; every document has `folder_path` as a FK. `acl_groups TEXT[]` on the folder row is copied into every chunk payload (`acl_groups` indexed field) and into `rag.documents.folder_path`.

8.1 Managing folders

```
rag-admin folders create /qms/normen --groups admin,qms,engineering --description "QMS:
rag-admin folders set    /qms/normen admin,qms,engineering,auditor
rag-admin folders move   /qms/normen /qms/standards      # rewrites rows; no re-embed
rag-admin folders delete /qms/normen --wait               # fails if docs still exist
```

8.2 ACL change propagation

- Qdrant **payload rewrite**, no re-embedding. Cheap — Qdrant updates the indexed `acl_groups` field in place.
- A `reacl` job runs per affected document; progress visible in `rag-admin jobs list`.
- DuckDB views (`views.v_<table>_<group_hash>`) are regenerated lazily on first access per user's group-set.

8.3 Read versus write (v1 limitation)

v1 lumps read and write access under "group membership". Anyone with read on a folder can also upload there (admins can always). v1.1 will split `acl_read` / `acl_write` lists — tracked as a known limitation.

8.4 Default ACL

New documents without an explicit folder default to `DEFAULT_FOLDER_ACL` (typically `admin`). The ingestion-api refuses to upload to a non-existent folder — this is a safeguard against accidentally-public files.

9. Document ingestion and lifecycle

9.1 States

```
queued → parsing → embedding → indexed      (happy path)
                                   → failed      (retryable up to 3x)
                                   → superseded   (soft-deleted or replaced)
```

Transitions are written atomically to `rag.documents.status`; a stuck `parsing` or `embedding` for > 10 min usually means a container crash — inspect `docker compose logs ingestion-worker`.

9.2 Supported formats

- **Text PDF** — fast path, structure preserved.
- **Scanned PDF** — OCR via EasyOCR (default) or Tesseract (`OCR_LANG=deu+eng`). Slower, quality OCR-bound. Target > 90 % text recovery on the fixture set.
- **DOCX** — sections, lists, tables.
- **XLSX** — every sheet embedded as chunks *and* loaded as a DuckDB typed table.
- Optional (enable per-format after Phase 5): **PPTX**, **HTML**, **MD**.

9.3 Bulk ingestion

```
mkdir -p drop/qms/normen
cp /Volumes/Share/QMS/*.pdf drop/qms/normen/

rag-admin docs ingest-dir drop/ --base-folder /
# or dry run:
rag-admin docs ingest-dir drop/ --base-folder / --dry-run
```

Progress on the **Ingestion** Grafana dashboard:

- queue depth
- throughput MB/s
- parse failures by mime type
- top failing documents by error code

9.4 Dedupe

SHA-256 of the file contents is the deduplication key. Same file to same folder → HTTP 409 with the existing `doc_id`; no re-parse, no re-embed. Same file to a different folder is a new document (different ACL).

9.5 Versioning

Uploading a same-name file with a different SHA-256 creates a new `doc_id` and marks the previous one `superseded`. The old chunks are dropped from the index; the old bytes live in MinIO under `raw/{old_id}/...` for 30 days.

9.6 Reindexing

Needed when:

- Chunking config changed (`CHUNK_MAX_TOKENS` , `PRESERVE_TABLES` , etc.).
- Parser upgrade that is expected to improve quality (run `scripts/eval.py` to measure).
- A source file was mutated in place (rare; prefer versioning).

```
rag-admin docs reindex <doc_id>
rag-admin docs reindex --folder /qms/normen
rag-admin docs reindex --all --confirm          # minutes to hours
```

9.7 Deletion

- **Soft delete** (default): marks `superseded` , removes from index; bytes kept 30 days in MinIO.
- **Hard delete**: `--hard` flag, requires `ADMIN_CONFIRM=yes` env to run. Deletes bytes + all rows + all Qdrant points. Irreversible.

9.8 Failed ingestions

- Password-protected PDFs: error message points to this; owner must provide an unlocked copy or admin enables `PDF_UNLOCK_ATTEMPTS=true` (empty-password attempt only).

- Docling parse errors: `PARSER_FALLBACK=pymupdf` re-runs through a lightweight text path for that doc only. Quality degrades; document it.
- Oversized files: increase `INGEST_MAX_BYTES` but monitor worker RAM.

10. Retrieval behaviour and tuning

10.1 The four query classes

Class	Example	Path	LLM tier
Lookup	"Welche Norm gilt für Typ-B-Schränke?"	Hybrid + rerank	Gemma 2 9B
Extraction	"Liste alle Lieferfristen aus Angebot-2024-09.pdf."	Hybrid + rerank, long context	Qwen 2.5 32B
Table-math	"Welches Projekt hatte 2024 die höchste Marge?"	Parallel SQL + hybrid for context	Qwen 2.5 32B
Synthesis	"Fasse unsere Position zu Thema X über alle QMS-Dokumente."	Hybrid + rerank top 20, map-reduce	Llama 3.3 70B

The classifier is a one-shot Gemma 9B call with a versioned prompt. Langfuse records the classification + confidence for every query — monitor drift on the Quality dashboard.

10.2 Retrieval algorithm (short version)

```
classify → optional rewrite (HyDE/paraphrase x2)
→ embed(query) dense + sparse
→ Qdrant prefetch: dense top 50 + sparse top 50, both with ACL filter
→ RRF fusion → top 50
→ TEI rerank → top 12
→ (if table-math) SQL branch in parallel: LLM → duckdb-api → rows
→ build prompt (DE/EN/bilingual)
→ Ollama stream with chosen tier
→ SSE tokens + citations event
→ audit + Langfuse trace
```

Every Qdrant call copies the same ACL filter — **there is one search code path**. This is a hard rule ([retrieval-agent brief](#)).

10.3 Knobs

Everything in `.env` :

Knob	Default	Effect	Requires reindex?
TOP_K_DENSE	50	Candidates from dense search	No
TOP_K_SPARSE	50	Candidates from sparse search	No
TOP_K_RERANK	12	Candidates after rerank → prompt	No
HYBRID_FUSION	rrf	or dbsf (Qdrant's Distribution-Based Score Fusion)	No
QUERY_REWRITE	true	Generates 2 paraphrases	No
CHUNK_MAX_TOKENS	512	Chunker upper bound	Yes
CHUNK_MIN_TOKENS	128	Chunker lower bound	Yes
PRESERVE_TABLES	true	Tables kept whole	Yes
CITATION_STYLE	brackets	[1] inline or footnotes	No
REFUSAL_ON_EMPTY	true	Never fabricate; refuse if no chunk supports	No

10.4 Prompt templates

Files on disk at `config/retrieval/prompts/{de,en,bilingual}.md`. Versioned like code. A runtime edit requires a `retrieval-api` restart — there is no hot admin UI for prompts in v1.

Every prompt change must show parity or uplift on the 50-query gold set (`scripts/eval.py`) before it ships. Baselines live in `docs/eval/baseline-YYYY-MM-DD.md`.

10.5 Refusal style

If the reranked top-K does not contain a chunk supporting the answer:

- DE: *"Ich habe dazu in den zugänglichen Dokumenten keine Information gefunden."*
- EN: *"I didn't find information on that in the documents you can access."*

No partial fabrication. No "maybe this, maybe that." The *refusal itself* is part of the product.

11. Model management

11.1 Model catalogue

Purpose	Default	Size	RAM	Tier env var
Fast LLM	<code>gemma2:9b-instruct-q5_K_M</code>	~7.5 GB	~7.5 GB	<code>LLM_FAST</code>
Reasoning LLM	<code>qwen2.5:32b-instruct-q4_K_M</code>	~20 GB	~20 GB	<code>LLM_REASONING</code>
Heavy LLM	<code>llama3.3:70b-instruct-q4_K_M</code>	~40 GB	~40 GB	<code>LLM_HEAVY</code>
Embedder	<code>bge-m3</code>	~1 GB	~1 GB	<code>EMBED_MODEL</code>
Reranker	<code>BAAI/bge-reranker-v2-m3</code>	~0.6 GB	~0.6 GB	<code>RERANK_MODEL</code> (TEI)

Ollama lazy-loads and evicts based on `OLLAMA_MAX_LOADED_MODELS` (default 2 on 64 GB).
`OLLAMA_KEEP_ALIVE=10m` means idle models are evicted after 10 minutes.

11.2 Router config

`config/retrieval/models.yaml` (owned by `llm-agent`) is the single place to change which model handles which class:

```
classes:
  lookup:      { model: gemma2:9b-instruct-q5_K_M,  max_tokens: 400 }
  extraction:  { model: qwen2.5:32b-instruct-q4_K_M, max_tokens: 1200 }
  table-math:  { model: qwen2.5:32b-instruct-q4_K_M, max_tokens: 800 }
  synthesis:   { model: llama3.3:70b-instruct-q4_K_M, max_tokens: 1600 }
  embedding:   { model: bge-m3, dimensions: 1024 }
  reranker:    { model: BAAI/bge-reranker-v2-m3, server: tei }
```

Restart: `docker compose restart retrieval-api`. No reindex required.

11.3 Changing models

```
# Pull a new variant
docker compose exec ollama ollama pull qwen2.5:32b-instruct-q5_K_M

# Edit models.yaml; restart
docker compose restart retrieval-api

# Evaluate against the gold set
python scripts/eval.py --gold config/eval/gold-queries.yaml

# Commit whichever has better recall@10 / citation accuracy
```

Keep the old model until the new one is validated; it's just disk.

11.4 Constrained-host fallback

`LLM_PROFILE=compact` drops the heavy tier and routes `synthesis` to `reasoning`. Acceptable on 48 GB hosts. Document it; the sizing matrix in [ADR-004](#) shows what fits where.

11.5 Embedding model change = breaking change

Changing `EMBED_MODEL` triggers:

1. A new Qdrant collection (vector shape differs → fresh collection).
2. A full reindex job on all documents.
3. A MAJOR stack version bump.

Plan a maintenance window. The new collection is created blue/green; requests keep hitting the old until the reindex finishes, then a cutover flag flips.

12. Observability

Three signals kept separate:

12.1 Langfuse — LLM traces

- URL: `https://$PRIMARY_DOMAIN/langfuse/`.
- One parent trace per query with child spans: `classify`, `rewrite`, `dense_search`, `sparse_search`, `rerank`, `sql_plan`, `sql_exec`, `generate`.
- Each span carries: model, inputs (truncated), outputs, latency, token counts, cost (unused but emitted).
- Filters for debugging:
 - `latency > 10s` — find slow queries
 - `retrieval.rerank_score_top < 0.4` — low-confidence retrievals
 - by user email — complaint triage

Replay from Langfuse UI with a different model for A/B comparisons.

12.2 Prometheus + Grafana — metrics

Four provisioned dashboards:

Dashboard	Key panels
Overview	QPS, p50/p95 latency by class, error rate, model in use, <code>rag_build_info{version}</code>
Ingestion	Queue depth, job state pie, throughput MB/s, parse failures by mime type
Infra	Container CPU/RAM, disk free, network, docker log volume
Quality	(populated from Langfuse exports) rerank uplift, refusal rate, top "no citation" queries

Key custom metrics:

- `rag_query_total{class}` — counter
- `rag_query_latency_seconds{phase,class}` — histogram
- `rag_retrieval_hits{source=dense|sparse|rerank}` — counter
- `rag_ingestion_jobs_total{state}` — gauge
- `rag_doc_count`, `rag_chunk_count` — Postgres-backed, refreshed every 60 s

12.3 Loki + Promtail — logs

- All container stdout is shipped to Loki.
- Retention: 14 days INFO, 90 days WARN+.
- Inspect via Grafana Explore → Loki datasource → `{container="retrieval-api"}`.

12.4 Alerts

Channels: `$_{DATA_ROOT}/alerts.log` always; optional webhook via `ALERT_WEBHOOK_URL` (Teams, Slack, Mattermost).

Default rules:

Rule	Threshold	Severity
<code>rag_disk_free_pct < 10</code>	persistent	critical
Any container unhealthy for > 5 min	—	critical
<code>rag_ingestion_queue_depth > 200</code>	—	warning
<code>rag_ingestion_queue_depth > 500</code>	—	critical
Backup not run in 26 h	—	critical
p95 latency on <code>lookup</code> > 8 s for 10 min	—	warning

Silence: `rag-admin alerts silence <rule> --until 2026-05-01`.

13. Backup and restore

13.1 What is backed up

Source	Method	Typical size
<code>postgres</code> (rag DB)	<code>pg_dump -Fc</code>	« 1 GB
<code>authentik-db</code>	<code>pg_dump -Fc</code>	< 100 MB
<code>langfuse-db</code>	<code>pg_dump -Fc</code>	1 GB/month growth
<code>minio</code>	<code>mc mirror</code>	≈ raw corpus size + 5 %
<code>qdrant</code>	snapshot API → tarball	scales with chunks
<code>duckdb</code>	file copy	tens of MB
<code>redis</code>	AOF file copy	< 10 MB

What is **not** backed up: Ollama model weights (re-pull from source), TEI/Docling caches (regenerated), Loki (ephemeral), Prometheus TSDB (ephemeral), Grafana state (provisioned from config).

13.2 Schedule

- Nightly at **02:15 local** via launchd (macOS) or systemd timer (Linux).
- Retention GFS: **7 daily / 4 weekly / 12 monthly**.
- Output: `${BACKUP_ROOT}/YYYY-MM-DD/`.
- Optional GPG encryption: set `BACKUP_GPG_PASSPHRASE_FILE`.

13.3 Manual run

```
rag-admin backup run
ls -lh "$BACKUP_ROOT/$(date +%F)/"
```

13.4 Restore rehearsal (mandatory)

```
make down
sudo mv /var/lib/reineke /var/lib/reineke.old
sudo mkdir /var/lib/reineke && sudo chown "$USER" /var/lib/reineke

rag-admin restore plan "$BACKUP_ROOT/2026-04-22/"
rag-admin restore apply "$BACKUP_ROOT/2026-04-22/"
make up
make wait-healthy
rag-admin query "Welche Norm gilt für Typ-B-Schränke?" # sanity
```

Rehearse at least once per major upgrade. A backup that has never been restored is a hope, not a backup.

13.5 Encryption at rest

- macOS: FileVault on the data disk.
- Linux: LUKS for `${DATA_ROOT}` or ZFS native encryption.
- Backup medium: GPG symmetric with a 25-char random passphrase; passphrase in password manager; path in `BACKUP_GPG_PASSPHRASE_FILE`.

14. Security operations

14.1 Trust model recap

- Only Caddy binds host ports.
- All service-to-service JWT validation goes through `services/common/auth.py` (shared library) hitting the Authentik JWKS endpoint. RS256, 2048-bit.
- Internal API calls use `INTERNAL_SERVICE_TOKEN` (single shared secret rotated with `scripts/rotate-secrets.sh`).
- Qdrant: API key, held only by the APIs.
- MinIO: IAM keys; browser uses pre-signed URLs.
- Ollama: no auth; relies on network isolation. Never bind to host.

14.2 Credential rotation

```
bash scripts/rotate-secrets.sh
```

Rotates Postgres passwords, Qdrant API key, MinIO root keys, internal service token. Services restart rolling. Takes < 1 min.

Authentik admin account: rotate via UI; new recovery secret into password manager.

14.3 TLS

Caddy auto-rotates the internal CA every 90 days. Distribute the new CA to clients when you rotate:

```
scripts/export-ca.sh > ca.crt  
# Distribute via MDM or manual
```

For Let's Encrypt via DNS-01, that is in the Caddyfile and handled automatically.

14.4 Incident response

Scenario	Action
Suspected credential leak	<code>rag-admin sessions revoke-all</code> ; <code>bash scripts/rotate-secrets.sh</code> ; audit last 30 days of <code>audit_log</code> for unusual queries; verify <code>ADMIN_BACKUP_TOKEN</code> has not been used without record
Host compromise suspected	<code>make down</code> ; image disk; restore from last clean backup onto a fresh host; rotate everything before re-onboarding users
User reports a wrong/leaky answer (PII, other-group content)	Snapshot the <code>audit_log</code> row + the source Qdrant point; verify the source document's folder ACL; consider a scoped delete or folder move

14.5 Audit export (GDPR)

```
rag-admin audit export --from 2026-01-01 --to 2026-03-31 --format csv > q1.csv
```

Fields defined in [docs/02_ARCHITECTURE.md §4](#). A data subject request for their own rows: filter by `user_id`.

15. Upgrades

15.1 Patch / minor (`v1.0.0` → `v1.0.1`)

```
git fetch && git checkout v1.0.1
make pull
make up
make wait-healthy
```

Read the changelog first for **migration** or **reindex** notes.

15.2 Major (v1.x.y → v2.0.0)

```
make backup
git checkout v2.0.0
make pull
rag-admin migrate preflight      # lists what must change
rag-admin migrate apply         # queues reindex jobs; watch Grafana Ingestion
```

Plan a 30–120 min window depending on corpus size. Embedding-model bumps are the most expensive — they trigger a full re-embed of every chunk.

15.3 Ollama model upgrades

```
docker compose exec ollama ollama pull qwen2.5:32b-instruct-q5_K_M
# Edit config/retrieval/models.yaml, then:
docker compose restart retrieval-api
python scripts/eval.py          # compare recall + citation accuracy
```

Only keep the winner.

16. Performance tuning

16.1 Latency budgets

Class	Target p95 to last token
lookup	≤ 5 s
extraction	≤ 15 s
table-math	≤ 20 s
synthesis	≤ 60 s

If you breach, check Langfuse traces for the worst span.

16.2 Common tuning moves

Symptom	Action	Cost
Lookup p95 drifting to 8 s+	Lower <code>CHUNK_MAX_TOKENS</code> to 300, reindex; confirm model not evicted	Reindex time
Low recall on narrative docs	Raise <code>CHUNK_MAX_TOKENS</code> to 800, reindex	Reindex time
Rerank is slow	Lower <code>TOP_K_DENSE</code> + <code>TOP_K_SPARSE</code> to 30 each	Minor recall hit
Synthesis too slow on M4 Max	<code>LLM_PROFILE=compact</code> (drop 70B)	Some quality loss on cross-doc synth
Memory thrash	Confirm <code>OLLAMA_MAX_LOADED_MODELS=2</code> , <code>OLLAMA_NUM_PARALLEL=1</code>	—
Qdrant RAM pressure	Enable/verify <code>quantization.scalar.type=int8</code> ; keep <code>on_disk_payload=true</code>	Negligible quality loss

16.3 Disk hygiene

- `docker system df` ; weekly `docker system prune -f --filter until=168h` .
- `ollama list` to inspect cached models; `ollama rm <old>` to free space.
- Prometheus TSDB retention is 15 days; raise only if you have the disk.

17. Troubleshooting

17.1 General method

1. `docker compose ps` — what's unhealthy?
2. `docker compose logs --since 30m <service>` — recent errors.
3. `rag-admin status` — app-level state (queue depth, doc count, last query).
4. Langfuse or Grafana for symptom-specific dashboards.

17.2 Symptom → cause table

Symptom	Likely cause	Next step	
Everything returns "information not available"	Missing <code>groups</code> claim in JWT	Authentik OIDC property mappings; <code>retrieval-api</code> logs show <code>groups=[]</code>	
Ingestion stuck at <code>parsing</code>	Docling exception (often encrypted PDF)	<code>docker compose logs ingestion-worker</code> ; retry after fix	
Ingestion stuck at <code>embedding</code>	Ollama cold-loading or OOM	<code>ollama ps</code> ; check RAM; lower <code>OLLAMA_MAX_LOADED_MODELS</code>	
"All queries slow"	Two heavyweight models loaded simultaneously	<code>ollama ps</code> ; lower concurrency; reduce <code>OLLAMA_KEEP_ALIVE</code>	
Answer cites the wrong chunk	Retrieval precision issue	Langfuse trace → rerank scores; if gold chunk absent, re-chunk / widen k; if present, reranker or language mismatch	
Answer in wrong language	Model language preference	Add "Answer in <lang>." to system prompt or user toggle	
Citation preview 404	MinIO pre-signed URL expired (> 1 h)	Regenerate by re-clicking; if persistent, MinIO down	
ACL leak (user saw other-group chunk)	Bug ; ACL filter missed on a search path	Snapshot trace, escalate to retrieval-agent; CRITICAL	
No backup last night	Backup script failed	Check <code>alerts.log</code> ; missed cron on macOS? <code>`launchctl list`</code>	grep reineke`
Qdrant collection corruption	Rare	Restore from snapshot (<code>scripts/qdrant-snapshot.sh apply <snap></code>); fallback is full reindex	

17.3 Escalation

Three repeat failures of the same acceptance criterion during a build → the coordinator halts and escalates. In production, three repeat runbook failures of the same cleanup → file a ticket, do not loop.

18. Extending the system

Documented extension points. Each is a bounded change with a known migration shape.

18.1 New mime type

1. Add parser branch in `services/docling/app.py`.
2. Add fixtures under `tests/fixtures/`.
3. Extend `SUPPORTED_MIME_TYPES` env in `ingestion-api`.
4. No schema change.

18.2 New LLM

```
docker compose exec ollama ollama pull <new-model>
# Edit config/retrieval/models.yaml
docker compose restart retrieval-api
python scripts/eval.py          # measure
```

18.3 New embedder (breaking)

- New Qdrant collection (different vector shape).
- Blue/green reindex.
- MAJOR version bump.
- Update `EMBED_MODEL` env + `models.yaml.embedding.dimensions`.

18.4 New ACL predicate (e.g. confidentiality level)

1. Postgres migration: add column on `rag.documents`.
2. Add Qdrant indexed payload field.
3. Extend the filter clause in the *single* retrieval code path.
4. Backfill script updates all existing documents.
5. Optional: UI exposure in admin view.

18.5 New data source (wiki, mail)

- Write a connector service that produces `DoclingDocument`-like JSON.
- Submit to `ingestion-api`'s existing endpoint with `folder_path` + ACLs.
- No retrieval changes required.

18.6 Two-host split (v1.1)

When corpus > 5 k docs AND team > 30 active users:

- Worker host: Ollama + TEI + ingestion-worker (heavy CPU/Metal).
- App host: everything else.
- Docker overlay network or WireGuard between them.
- `scripts/split-to-two-hosts.sh` generates the override file. Rehearse in staging.

19. Appendices

19.1 Internal port map

Service	Port	Notes
caddy	80, 443 (host)	Only host-bound
authentik-server	9000	UI + OIDC
postgres	5432	rag DB
redis	6379	queue + pub/sub
minio	9000 (S3), 9001 (console)	IAM creds
qdrant	6333 (REST), 6334 (gRPC)	API-key auth
ollama	11434	no auth
tei-reranker	8080	internal
docling	8001	internal
ingestion-api	8010	JWT
retrieval-api	8020	JWT
duckdb-api	8030	JWT
openwebui	8080	proxied
pipelines	9099	internal
langfuse	3000	proxied under /langfuse
prometheus	9090	internal
grafana	3000	proxied under /grafana
loki	3100	internal

19.2 Capacity planning (rule of thumb)

Corpus	Qdrant disk	Qdrant RAM (int8)	MinIO disk	Postgres
500 docs (~50k chunks)	~0.5 GB	~0.2 GB	raw × 1.05	~0.5 GB
2 000 docs (~200k chunks)	~2 GB	~0.8 GB	raw × 1.05	~1.5 GB
10 000 docs (~1M chunks)	~10 GB	~4 GB	raw × 1.05	~8 GB
50 000 docs (~5M chunks)	~50 GB	~20 GB	raw × 1.05	~40 GB

Ollama model ceiling: ~80 GB (all tiers + embedder + reranker). Keep ≥ 20 % headroom on disk.

19.3 Useful one-liners

```
# Count indexed chunks
docker compose exec postgres psql -U rag -c "select count(*) from rag.chunks;"

# Top 10 slow queries, last 24h
docker compose exec postgres psql -U rag -c "
  select substring(query,1,80), latency_ms
  from rag.audit_log
  where ts > now() - interval '24 hours'
  order by latency_ms desc limit 10;"

# Qdrant sanity
curl -s -H "api-key: $QDRANT_API_KEY" http://localhost:6333/collections/chunks | jq .

# Active sessions (last 5 min)
docker compose exec postgres psql -U rag -c "
  select count(distinct user_id) from rag.audit_log
  where ts > now() - interval '5 minutes';"

# Who's loaded in Ollama
docker compose exec ollama ollama ps
```

19.4 Decommissioning

```
make down
rag-admin backup run # final
# Archive ${BACKUP_ROOT} to cold storage; verify checksum
# Remove ${DATA_ROOT}
# Revoke Authentik OIDC clients
# Remove DNS + TLS cert distribution
```

19.5 When to escalate to the build team

- Any acceptance-criterion regression after an upgrade (A1.1–A10.1 in [docs/05_IMPLEMENTATION_PLAN.md](#)).
- Confirmed ACL leak (critical; must re-open the retrieval-agent lane).
- Docling produces materially worse parses after an upgrade — run `scripts/eval.py` and attach the diff.
- Proposal to add a new container or change an ADR decision — requires a new ADR superseding the old; do not deploy around it.

Related documents in this repository:

- [TECH_DESCRIPTION.md](#) — conceptual overview, ADR index
- [USER_HANDBOOK.md](#) — handbook for end users
- [docs/04_OPERATIONS.md](#) — authoritative operations reference (this handbook is its working companion)
- [docs/02_ARCHITECTURE.md](#) — authoritative architecture reference
- [docs/adr/](#) — decision rationale